

Requirements Engineering of Industrial Automation Systems

Adapting the CESAR Requirements Meta Model for Safety-Critical Smart Grid Software

Roopak Sinha¹, Sandeep Patil², Cheng Pang³, Valeriy Vyatkin^{2,3}, and Barry Dowdeswell¹

¹School of Computer and Mathematical Sciences, Auckland University of Technology, Auckland, New Zealand

²Department of Computer Science, Electrical and Space Engineering, Luleå University of Technology, Luleå, Sweden

³Department of Electrical Engineering and Automation, Aalto University, Finland

roopak.sinha@aut.ac.nz, {sandeep.s.patil, cheng.pang.phd, vyatkin}@ieee.org, barry.dowdeswell@aut.ac.nz

Abstract—Requirements engineering is the first stage in the development of any system. For safety-critical industrial systems like smart-grids, we must ensure that requirements are properly elicited, defined, analyzed and managed. This paper adapts the requirements framework developed in the CESAR European project, called the CESAR requirements meta-model, to support all aspects of requirements engineering for safety-critical systems. This enables the formalization of requirements in order to automate and assist in many aspects of later stages in the system development life cycle. Using a smart grid system from the FREEDM project, we illustrate that the adapted requirements engineering framework is comprehensive and rich for large safety-critical systems. We find that the use of the IEC 61499 function block standard provides an appropriate system modelling and implementation framework, which complements the strengths of the proposed requirements engineering framework. Traceability links between requirements and components of an IEC 61499 system model can help with automatic test case generation and formal analysis of requirements.

Keywords—requirements engineering; requirements system models; requirements specification language; IEC 61499; CESAR; FREEDM; smart grids; traceability

I. INTRODUCTION

Smart grids modernize traditional electrical grids by automating the gathering of information from various parts of the grid (consumers, suppliers, zones, etc.) and making intelligent decisions on how to drive these parts to achieve higher efficiency and reliability. The brain of a smart grid is intelligent distributed software, the design of which should prioritize large-scale safety, compliance, and re-configurability. The FREEDM project [1] aims to infuse smart grid capabilities within the electrical grid of the USA and identifies the IEC 61499 standard [2, 3] as a suitable design and development language for intelligent smart grid software.

Large-scale smart grid software must conform to a substantial number and types of requirements [4]. These include functional requirements relating to system features and functions, as well as non-functional requirements like safety, reliability, re-configurability, etc. Strong processes are followed in industry to ensure that systems conform to given requirements. However, the area of *requirements engineering* that precedes the design and development phases in the software development life cycle (SDLC) [5, 6] has received little attention in the context of large-scale safety-critical industrial soft-

ware systems such as for smart grids. Requirements engineering failures can be spectacular, e.g., the HMS Titanic's sinking in 1914 due to wrong steel plating requirements [7] or Ford's Edsel automobile's market failure due to a failure to capture end-users' requirements [8]. For safety-critical systems like smart grids, failures can have prohibitively expensive and dangerous consequences, resulting in a clear need to ensure that requirements engineering follows best-practice.

There has been some work in the area of requirements engineering for smart grids. In [9], ICT architectures for smart grids are examined and in [10], authors present security requirements with smart grid as a case study. Some works like [11] list some important cyber security and communication requirements for smart grids. Currently, CASE tools [12] are used for some requirements engineering aspects of industrial automation software. In [13], formal modelling and verification are used to model and test safety requirements. However, there is a lack of a unifying framework for requirements engineering of safety-critical industrial systems. Existing tools only support specific aspects of SDLC stages like design, development, testing, and deployment.

This paper employs the requirements engineering framework from the CESAR project [14]. In CESAR, the focus was creating tools, frameworks, and processes for developing large, distributed, and safety-critical systems. The CESAR meta-model is a unifying framework for developing large-scale safety-critical systems for automotive, rail, and aerospace sectors. The meta-model contains a requirements meta-model (RMM) to deal with requirements and a system meta-model to capture system and component models. The meta-models define domain-independent terminology, elements, and relationships to allow information exchange. Following a V-model of construction [15], requirements engineering enabled by the RMM is followed by the construction of a system model which is then realized into an implementation. The automation domain was studied peripherally in CESAR, and IEC 61499 was not studied at all. IEC 61499 can be shown to be a suitable and consistent model for developing a system using CESAR's principles due to its support for abstraction and incremental refinement following a V development approach. This paper adapts the RMM for the requirements engineering of safety-critical IEC 61499 software.

The RMM is a model of the requirements domain and serves as a unifying meta-model when requirements may be specified in different requirement specification languages

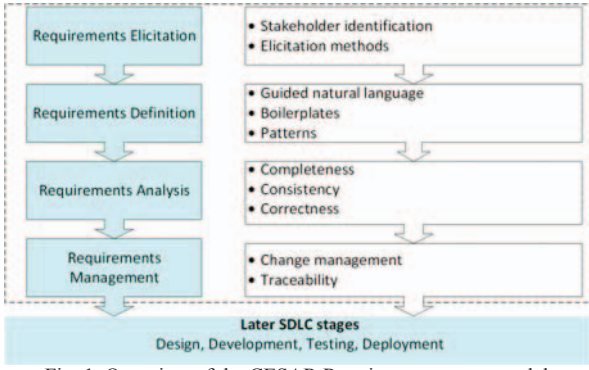


Fig. 1. Overview of the CESAR Requirements meta-model.

(RSLs). CESAR proposes its own RSLs for capturing both functional and non-functional requirements, supporting a seamless transition from informal to formal requirements. Formalized requirements make the RMM especially suitable for safety-critical systems. Fig. 1 shows the sub-stages of requirements engineering, and key processes and frameworks that we adapt and apply to smart-grid systems. These stages allow gradual, semi-automatic formalization of requirements, and consequently their efficient uses later on in the SDLC.

The key contributions of this article are:

1. We present the first comprehensive framework for requirements engineering of safety-critical IEC 61499 systems based on the CESAR RMM.
2. We show that the proposed framework can assist in automating many aspects of later stages in the SDLC while using IEC 61499 as the system modelling platform.
3. Using a smart grid system from FREEDM, we illustrate how the proposed requirements engineering framework can be applied to safety-critical IEC 61499 software.

The rest of this paper is organized as follows. Section II introduces the smart grid case study. Section III presents the proposed requirements engineering framework. Section 0 shows how the adapted framework can be used in later stages of the SDLC. Concluding remarks appear in Section V.

II. MOTIVATING EXAMPLE

The FREEDM system [1] is a prototype of reasonably complex smart grid with modular architecture. Fig. 2.(a) shows FREEDM's zone concept protection scheme, where the grid is divided into zones using fault isolation devices or new generation circuit breakers. These devices are placed at the borders of two zones and can isolate one zone from the other if needed. Within a zone, devices such as renewable energy resources, storage devices, smart meters, and loads are grouped and managed by a solid-state transformer (SST) block and encapsulated as a distributed grid intelligence node.

Fig. 2.(a) divides the protection scheme into three zones 1-3 with an overall zone 0 serving as backup protection. Fig. 2.(b) shows one zone with energy resources, SSTs, and some load. In each zone, a merging unit placed at the terminal of distribution lines and the feeders of the load measures, digitizes, and transfers the current values to the zone's controller, called an Intelligent Fault Management (IFM) device. The

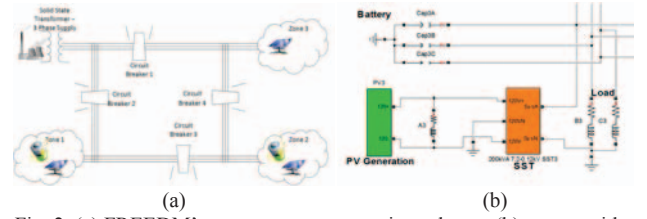


Fig. 2. (a) FREEDM's zone concept protection scheme, (b) a zone with energy storage device, energy resource, SST, and load.

IFM runs the protection algorithms and acts as a local node in the distributed smart grid intelligence.

The overall protection scheme inside an IFM consists of primary and secondary protection. Primary protection uses a differential scheme: if the sum of currents (in and out flows) in a zone equals zero this indicates either no fault or a fault outside the zone. In case the sum is not zero then the fault is within the zone and IFM may make the decision to trip the circuit breaker at the border with another zone (Eq.(1)).

$$\forall i \sum_j (in_i^j + out_i^j) = 0 \quad (1)$$

where $i = \text{zone and } j = \text{current in the zone}$

The secondary (overcurrent) protection in zone 0 examines the current values for zones 1, 2, and 3. The incoming current sample value for a zone is compared with pre-set current values (usually 3-5 times of the rated current for the zone). In case the zone 0 IFM finds the current for a zone to be above its threshold, a fault is identified in that zone and the corresponding circuit breaker is activated (Eq.(2)).

$$\forall i \text{ current}_i^j > \varepsilon \Rightarrow \text{fault}_i \quad (2)$$

where $i = \text{zone, } j = \text{current in the zone}$

and ε is the threshold

An IEC 61499 implementation of this control scheme can be found in [16]. If such a system is to be developed from scratch, a number of requirements engineering issues must be addressed. Firstly, all requirements must be elicited and captured. Concepts such as zones, IFMs, protection algorithms, etc., and their interrelationships must be understood and documented. Also, captured requirements should be readily usable for later stages in the SDLC. For example, a requirement that a zone correctly implements its protection scheme must ideally be checked automatically using formal verification [17] [18]. During the SDLC, requirements and interrelationships must be managed and links to design artefacts they correspond to must also be maintained. The lack of a comprehensive requirements engineering framework makes these issues very difficult to address. However, adapting the CESAR RMM can address many of these issues very efficiently.

III. REQUIREMENTS ENGINEERING FOR SAFETY-CRITICAL INDUSTRIAL AUTOMATION SOFTWARE

A. Requirements Elicitation

The first stage in the requirements engineering process involves eliciting requirements for the system to be built. At first, we must identify all stakeholders – parties that will

TABLE I. SUBSET OF ELICITED REQUIREMENTS FOR THE SMART-GRID SYSTEM.

ID	Description
R1	The system shall have 4 isolated fault tolerant zones.
R2	Each IFM controller should support a differential protection scheme (eq.1) of its zone.
R3	Zone 0 controller should support overcurrent protection scheme (eq.2) of the entire system.
R4	A healthy zone shall not become de-energized due to a fault within another zone. The IFM nodes should identify fault and isolate it from the rest healthy system. If primary protection fails, the secondary protection should act.
R5	Eliminate or make the IFM protection system resistant (as much as possible) to the undesirable operation of the energy storage device within a zone following the neighboring zone isolation.
R6	The IFM node shall not terminate or stall due to the lack of incoming (sampling) data (pause in the input stream). IFM node should be resistant against it and notify SCADA operator.
R7	The IFM node shall not terminate or stall due to invalid input. IFM node shall reject invalid inputs and notify SCADA operator when an invalid input is received.
R8	The failure of an IFM node (either error in execution or physical device failure) shall not leave the zone unprotected.
R9	The central (zone 0) IFM should be deployed over at least 2 SSTs.

somehow interact with the system to be built. Stakeholders for a smart grid will include the client who commissions the work, government bodies, relevant safety and development standards like IEC 61850 [19, 20] and IEC 61499 [21, 22], grid suppliers and consumers, etc. Once all key stakeholders have been identified, requirements may be elicited using techniques such as single-person or group interviews, developing use-case and misuse-case scenarios, quality models to help specify quality requirements, and developing prototypes for early validation. Elicitation may also involve analyzing the domain and domain-specific terminology and artefacts. Also, requirements captured at this stage can be assumptions, goals (system objectives), or requirements.

While elicitation would generally result in hundreds (or thousands) of requirements, we consider a small subset of elicited requirements for the smart grid system, as shown in Table 1. These requirements exhibit the typical characteristics of elicited requirements: ambiguous, informal, and possibly unclear. For example, while requirement **R2** demands that each zone supports differential protection, the time intervals at which current in and out flows are sampled are left unclear. These aspects may be elaborated further in later stages of requirements engineering. While requirements **R2** and **R3** are functional requirements, requirement **R9** is a non-functional requirement relating to redundancy and safety.

B. Requirements Definition

Requirements definition uses appropriate requirements specification languages (RSLs) to formally describe elicited requirements. The RSLs included within the CESAR framework are guided natural language, boilerplates, and patterns.

Guided natural language: Elicited requirements such as those listed in Table 1 are sentences written in unconstrained natural language. Guided natural language requirements involves developing a *dictionary* of terms and keywords contained within the informal requirements. Guided natural language allows the flexibility of free text descriptions without adding any formal constraints to sentence formation. The focus here is the creation of a dictionary to interpret free-form sentences formally. Requirement **R1**, “The system shall have 4 isolated fault tolerant zones”, requires a dictionary defining

the terms *system*, *zone*, *fault tolerant*, and *isolated* to be interpreted. The GNLQ tool [23] can assist in developing a dictionary and can point out undefined concepts. The problem here is that badly formed requirements may require many more concepts to be defined than necessary.

Boilerplates: boilerplates are semi-complete natural language sentences that are completed to form requirements [24]. Consider for example the boilerplate:

B1: The <system> shall contain <number> of <system>.

The incomplete parts of a boilerplate refer to system attributes. <system> and <number> are placeholders referring to a system or component and a positive number, respectively. For the smart grid system described in Fig. 2., we can instantiate boilerplate **B1** to the following requirement.

RB1: The *smart grid* shall contain 4 of *zone*.

While they are not as flexible as guided natural language, boilerplates are usually easier to use. Some informal requirements need to be elaborated in order to fit into given boilerplates. Consider again requirement **R1**. It can be broken down into two parts: “The smart grid shall contain 4 zones” (same as **RB1** above), and “A zone should be fault tolerant and isolated”. The second part of the requirement can be created as two instances of the following boilerplate

B2: A <system> should always be <quality attribute>.

The two instantiated requirements become:

RB2_1: A *zone* should always be *fault tolerant*.

RB2_2: A *zone* should always be *isolated*.

We can combine requirements using an “and” boilerplate:

B3: <boilerplate> and <boilerplate>

We now form the requirements:

RB3_1: **RB1** and **RB2_1**

R1 = RB3_2: **RB3_1** and **RB2_2**

Here, **RB3_2** is a boilerplate version of requirement **R1**.

Pattern based RSL: Patterns use a stronger formalism for requirements in CESAR than boilerplates and feature a fixed

semantics, allowing easier analysis and validation. However, a stronger formalism does reduce the ease with which requirements are captured. A pattern may fall into one of many categories, like functional, probability, architectural, etc. Like boilerplates, patterns are incomplete but structured sentences, as exemplified below.

P1: *whenever* *Subject1* [**does not**] *emit[s]* *Event1* [*within Interval*], *Subject2 emits* *Event2* [*within Interval*]

In this pattern, bold words like **whenever** represent keywords, phrases like [**does not**] in square brackets are optional, and italicized elements like *Subject1* and *Event1* represent system attributes like events, subjects, conditions, intervals, modes, etc. We can re-write part of requirement **R6** as:

RP1: *whenever* Environment **does not emit** SamplingData *within* 5 seconds, IFMNode **emits** ReportNoInputtoSCADA

For all three requirement types in CESAR, an underlying dictionary or *ontology* plays an important role in understanding system attributes, elements and their relationships, as well as classification viewpoints and domain-specific information. Developing an ontology is a part of requirements definition. The DODT tool [25] along with GNLQ can assist in creating an ontology by highlighting missing concepts and creating relationships or links between different concepts.

CESAR also supports a semi-automatic process to convert natural language requirements to boilerplates and boilerplates into patterns. The DODT tool automatically parses natural language sentences into attributes types such as users, systems, and actions, and also supports typographical correction, word substitution, and boilerplate. Fig. 3 shows the conversion of requirement **R1** (referred to as **REQ1** in the figure) into a boilerplate-based requirement. Similar tool support is available for turning boilerplates into patterns [14].

C. Requirements Analysis

Requirements must be complete, correct and consistent (CCC) [15]. Completeness ensures that requirements cover all needs and constraints for the system to be developed. Correctness ensures that requirements accurately reflect actual system needs and constraints. Consistency deals with removing conflicts, and allows a clear understanding of the system.

The DODT tool supports automatic requirements analysis. Fig. 4 shows the analysis results after the initial elicitation and design phase. DODT reports 100% completeness for the set of requirements. As the initial ontology contains only some concepts (like system, zone, and IFM), this is a false positive because many concepts and interrelationships are yet to be defined. The tool also finds an inconsistency between requirements relating to differential protection (**R2**). While one requirement states that differential protection must be triggered when Eq. (1) is not true, a second refined requirement states that differential protection must be triggered when Eq. (1) is

ID	Text	BP/NL	Trace From
REQ1	<u>System contains 4 IFM</u>	NL	
REQ1-BP	<u>System contains 4 IFM.</u>	BP	REQ1

Fig. 3. Tool-assisted formalization of requirements.

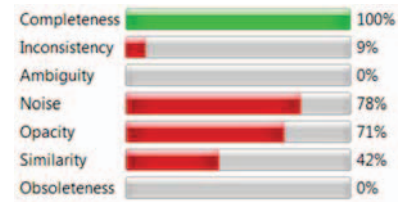


Fig. 4. Requirements forest for requirements **R1**, **R2**, **R3**, and **R9**.

not true in 20 time samples. These requirements are inconsistent with when protection is triggered. As expected, DODT also reports a large degree of noise and opacity as many concepts (like isolation, data, etc.) and interrelationships (like the link between IFM and invalid input) have not been added to the ontology. DODT also raises a concern that multiple requirements relate to the same set of concepts (similarity). These are mostly false negatives, as many requirements and sub-requirements typically target the same concepts.

D. Requirements Management

Requirements management involves organizing requirements and maintaining them during requirements engineering as well as subsequent stages in the SDLC. A key aspect of management is the organization of requirements. Elicited requirements are refined after elaboration and we must create links between existing and refined requirements. Furthermore, we may decompose requirements into sub-requirements and these links must also be preserved. We organize requirements into trees (directed, acyclic graphs) where an edge links a requirement to a sub-requirement. The end result is a forest, where each tree relates to an early elicited requirement refined by sub-requirements or more formal requirements.

Fig. 5 shows part of the requirements forest created from the requirements in Table 1. Requirement **R1** is refined from a natural language requirement to a boilerplate requirement **RB3_2**, as shown in Section 3B. In the corresponding tree, **R1** serves as the root node (bold) with **R3_2** as the sole child node. The combined requirement for system-wide protection is broken down into differential and overall protection requirements **R2** and **R3**. These two children of the root node are first refined with the descriptions (eq. (1) and (2)) and then further by adding information about how many times violations of the corresponding equations must be sampled before circuit breakers are tripped. Similarly, natural language based configuration requirement **R9** is converted into a formal requirement constraining the configuration of zone 0 to available SSTs using the Z3 SMT solver [26].

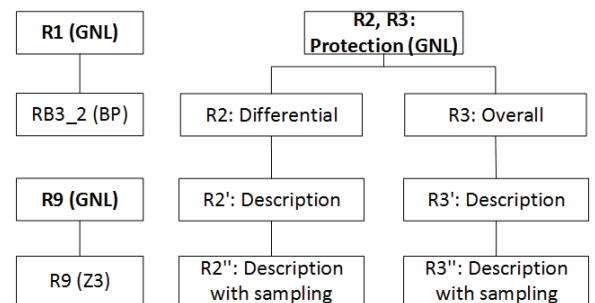


Fig. 5. DODT-based requirement analysis for the smart-grid system.

IV. USING REQUIREMENTS DURING IEC 61499 SDLC

Once requirements are captured, they can be used throughout the development cycle of a system. We present some cases where the proposed framework can significantly help improve the design process. In our work, all aspects of the system specification for the smart grid system can be done via IEC 61499, as most toolchains for IEC 61499 support design, development, testing and deployment. An IEC 61499 system is essentially a finitely nested network of **function blocks** [2]. Networks of function blocks are called composite blocks while the elementary building blocks are called basic blocks. During the *design* phase, a designer would usually start with architectural features or quality attributes like re-configurability, redundancy, etc., and then use functional requirements to decompose the system into blocks and then create the interfaces of the various blocks. An IEC 61499 system can be represented as a top-down component tree, with the system as the root node and sub-blocks as child nodes. The leaf nodes of this tree are essentially basic blocks [26]. Fig. 6 shows the component tree for the IEC 61499 design of the smart-grid system. While the actual system contains hundreds of blocks, we abbreviate the tree to highlight the concepts discussed in this paper. The system contains four zones, and zones 1, 2 and 3 are constructed using the same reusable (composite) function block. At the bottom of the figure are SSTs over which the software for each zone is configured to execute. As per requirement **R9**, zone 0 must be deployed over at least two SSTs to ensure sufficient overall protection even if an SST fails.

A key part of the design process is to ensure that *traceability* links are correctly made and maintained between every requirement and the set of blocks that must satisfy it when implemented. Creating traceability links at the design stage ensures that satisfaction of requirements is easier to check at a later stage. Fig.7 shows the traceability link between the abbreviated requirements forest and component tree of the smart grid system. Requirement **R1** must be satisfied by the system, and consequently all its sub-requirements must also be satisfied at that level unless otherwise indicated by the designer. On the other hand, the two children **R2** and **R3** of the combined requirement **R2** and **R3** must be satisfied by zone 0, and zones 1-3, respectively. Note that **R3** is linked to three system blocks. In general it is possible for requirements and blocks to have many-to-many links. Finally, requirement **R9** relating to

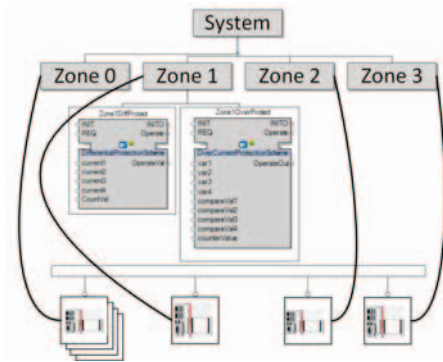


Fig.6. Component model for the IEC 61499 system for the smart grid.

redundancy of configuration in zone 0 is linked to the smart-grid system deployed over available SSTs.

During the *development* phase, programmers code the concrete functionality of individual design elements (blocks), and add any missing or previously undiscovered functionality. The proposed framework can help prevent scope creep, where programmers may be tempted to add more functionality than what is required by an element. Traceability links created during the design phase can be used for this purpose to ensure development is efficient.

During *testing*, a number of strategies are used to ensure that developed components, sub-systems, and the system satisfy key requirements. Functional requirements are often checked using simulation, formal verification [17, 27, 28], or by generating test cases. The proposed model allows automatic test case generation [29] as requirements are formalized and link to specific components. Formal verification is carried out for safety-critical aspects of a system, and can be enabled by using formalized pattern-based requirements. Similarly, simulation-based testing can be done through all levels of the system’s hierarchy, and semi or fully formalized requirements can help remove ambiguities. Finally, at *deployment*, we can automatically enforce configuration requirements like **R9** to ensure that the system is correctly configured, and re-configured if there is any change in the status of the SSTs [26].

V. DISCUSSION AND CONCLUSIONS

Most research works in safety-critical IEC 61499 systems are focused towards ensuring that safety requirements are met

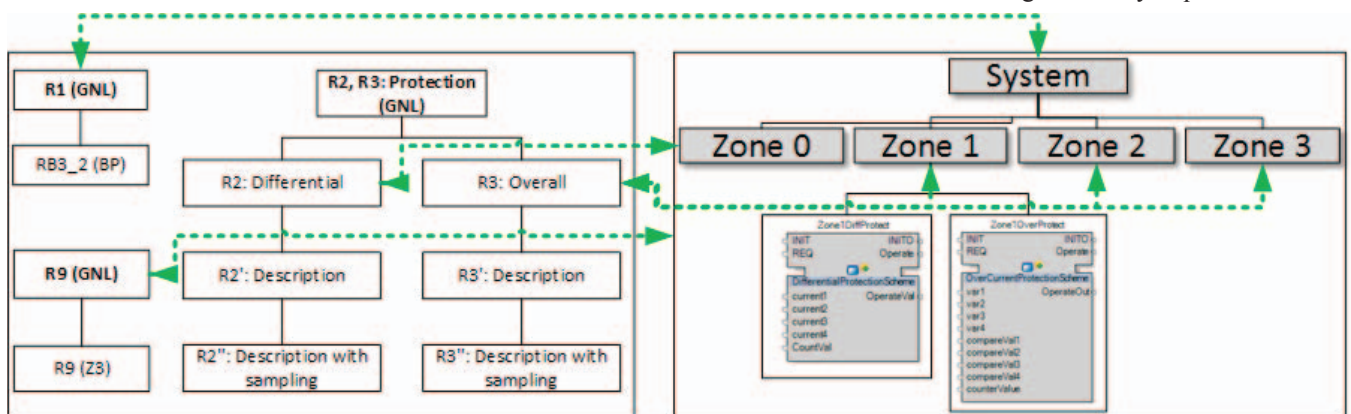


Fig.7. Traceability between requirements and system blocks.

via the use of appropriate tools. However, little attention has been given to creating a comprehensive requirements engineering platform for eliciting, capturing, analyzing and managing requirements. This paper fills this particular gap, by adapting the CESAR requirements meta-model to create a coherent requirements engineering framework for safety-critical IEC 61499 software. The modularity of IEC 61499 systems makes traceability easier and can help in subsequent phases like design, development, testing, and deployment. The adapted framework also allows automating and formalizing many steps in these stages of the design cycle.

There are several future directions for this work. Creating a set of most-used boilerplates and domain-specific ontologies will cut down enormously the time it takes to define and analyze requirements. Some parts of requirements management such as dealing with changes mid-way through the SDLC by carrying out impact analysis and finding affected requirements and system blocks needs more attention too. An interesting research topic is automatically creating traceability links when system blocks or requirements are refined.

ACKNOWLEDGMENT

This work was partially supported by SAUNA, S-STEP, and FREEDM projects. It was also supported by Wallenberg Foundation "Jubileumsanslaget" travel grants from Luleå University of Technology, and a travel grant from the Software Engineering Research Lab (SERL) at Auckland University of Technology, New Zealand.

REFERENCES

- [1] A. Q. Huang, M. L. Crow, G. T. Heydt, J. P. Zheng, and S. J. Dale, "The Future Renewable Electric Energy Delivery and Management (FREEDM) System: The Energy Internet," *Proceedings of the IEEE*, vol. 99(1), pp. 133-148, 2011.
- [2] "Programmable Logic Controllers — Part 3: Programming Languages, IEC Standard 61131-3," Third ed, 2013.
- [3] V. Vyatkin, "IEC 61499 as Enabler of Distributed and Intelligent Automation: State-of-the-Art Review," *IEEE Transactions on Industrial Informatics*, vol. 7(4), pp. 768-781, 2011.
- [4] G. Zhabelova, V. Vyatkin, and V. N. Dubinin, "Toward Industrially Usable Agent Technology for Smart Grid Automation," *Industrial Electronics, IEEE Transactions on*, vol. 62(4), pp. 2629-2641, 2015.
- [5] I. Jacobson, G. Booch, J. Rumbaugh, J. Rumbaugh, and G. Booch, *The unified software development process* vol. 1: Addison-Wesley Reading, 1999.
- [6] "ISO/IEC/IEEE Standard for Systems and Software Engineering - Software Life Cycle Processes," *IEEE STD 12207-2008*, pp. c1-138, 2008.
- [7] K. Felkins, H. P. Leigh, Jr., and A. Jankovic, "The royal mail ship Titanic: Did a metallurgical failure cause a night to remember?," *JOM*, vol. 50(1), pp. 12-18, 1998/01/01 1998.
- [8] A. T. Bahill and S. J. Henderson, "Requirements development, verification, and validation exhibited in famous failures," *Syst. Eng.*, vol. 8(1), pp. 1-14, 2005.
- [9] S. Rohjans, C. Danekas, and M. Uslar, "Requirements for Smart Grid ICT-architectures," in *Innovative Smart Grid Technologies (ISGT Europe), 2012 3rd IEEE PES International Conference and Exhibition on*, 2012, pp. 1-8.
- [10] N. Zafar, E. Arnautovic, A. Diabat, and D. Svetinovic, "System Security Requirements Analysis: A Smart Grid Case Study," *Systems Engineering*, vol. 17(1), pp. 77-88, 2014.
- [11] G. N. Ericsson, "Cyber Security and Power System Communication - Essential Parts of a Smart Grid Infrastructure," *Power Delivery, IEEE Transactions on*, vol. 25(3), pp. 1501-1507, 2010.
- [12] A. F. Case, "Computer-aided software engineering (CASE): technology for improving software development productivity," *SIGMIS Database*, vol. 17(1), pp. 35-43, 1985.
- [13] F. Bitsch, "Safety Patterns — The Key to Formal Specification of Safety Requirements," in *Computer Safety, Reliability and Security*, vol. 2187, U. Voges, Ed., ed: Springer Berlin Heidelberg, 2001, pp. 176-189.
- [14] A. Rajan and T. Wahl, *CESAR: Cost-efficient Methods and Processes for Safety-relevant Embedded Systems*: Springer, 2013.
- [15] K. Forsberg and H. Mooz, "The Relationship of System Engineering to the Project Cycle," *INCOSE International Symposium*, vol. 1(1), pp. 57-65, 1991.
- [16] G. Zhabelova, V. Vyatkin, and N. C. Nair, "Standard-based engineering and distributed execution framework for intelligent fault management for FREEDM system," in *IECON 2011 - 37th Annual Conference on IEEE Industrial Electronics Society*, 2011, pp. 2724-2729.
- [17] Z. E. Bhatti, R. Sinha, and P. S. Roop, "Observer based verification of IEC 61499 function blocks," in *Industrial Informatics (INDIN), 2011 9th IEEE International Conference on*, 2011, pp. 609-614.
- [18] S. Patil, V. Dubinin, C. Pang, and V. Vyatkin, "Neutralizing Semantic Ambiguities of Function Block Architecture by Modeling with ASM," in *Perspectives of System Informatics*, vol. 8974, A. Voronkov and I. Virbitskaite, Eds., ed: Springer Berlin Heidelberg, 2015, pp. 76-91.
- [19] G. Zhabelova, Y. Chen-Wei, and V. Vyatkin, "SysGrid: IEC 61850/IEC 61499 based engineering process for Smart Grid automation design," in *Industrial Informatics (INDIN), 2013 11th IEEE International Conference on*, 2013, pp. 364-369.
- [20] Y. Chen-Wei, V. Vyatkin, A. Mousavi, and V. Dubinin, "On automatic generation of IEC61850/IEC61499 substation automation systems enabled by ontology," in *Industrial Electronics Society, IECON 2014 - 40th Annual Conference of the IEEE*, 2014, pp. 3577-3583.
- [21] T. Strasser, F. Andren, J. Kathan, C. Cecati, C. Buccella, P. Siano, *et al.*, "A Review of Architectures and Concepts for Intelligence in Future Electric Energy Systems," *Industrial Electronics, IEEE Transactions on*, vol. 62(4), pp. 2424-2438, 2015.
- [22] P. Vrba, V. Marik, P. Siano, P. Leitao, G. Zhabelova, V. Vyatkin, *et al.*, "A Review of Agent and Service-Oriented Concepts Applied to Intelligent Energy Systems," *Industrial Informatics, IEEE Transactions on*, vol. 10(3), pp. 1890-1903, 2014.
- [23] I. Omoronyia and T. Stålhane. (2012). *GNLQ - Guided Natural Language and Requirement Boilerplates*. Available: <http://gnlq.sourceforge.net/>
- [24] E. Hull, K. Jackson, and J. Dick, "DOORS: a tool to manage requirements," in *Requirements engineering*, ed: Springer, 2002, pp. 187-204.
- [25] S. Farfeleder, T. Moser, A. Krall, T. Stålhane, H. Zojer, and C. Panis, "DODT: Increasing requirements formalism using domain ontologies for improved embedded systems development," in *Design and Diagnostics of Electronic Circuits & Systems (DDECS), 2011 IEEE 14th International Symposium on*, 2011, pp. 271-274.
- [26] R. Sinha, K. Johnson, and R. Calinescu, "A Scalable Approach for Re-Configuring Evolving Industrial Control Systems," in *19th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA 2014)*, Barcelona, Spain, 2014, pp. 1-8.
- [27] S. Patil, V. Dubinin, and V. Vyatkin, "Formal Verification of IEC61499 Function Blocks with Abstract State Machines and SMV - Execution Semantics," in *Symposium on Dependable Software Engineering*, Nanjing, China, 2015.
- [28] S. Patil, V. Dubinin, and V. Vyatkin, "Formal Verification of IEC61499 Function Blocks with Abstract State Machines and SMV - Modelling," in *The 13th IEEE International Symposium on Parallel and Distributed Processing with Applications (IEEE ISPA-15)*, Helsinki, Finland, 2015.
- [29] T. Hussain and G. Frey, "UML-based Development Process for IEC 61499 with Automatic Test-case Generation," in *11th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA 2006)*, Prague, Czech Republic, 2006, pp. 1277-1284.